Phishing URL Detection Project Using Python

Program and Output Report

Date: 21 October 2026

1. Project Overview

This project detects potentially phishing URLs using Python. The program analyzes a URL using rule-based checks and calculates a risk score. If the score is 3 or more, the URL is marked as potentially suspicious.

2. Detection Rules

Detection Rule	Risk Points
URL longer than 100 characters	1
URL uses HTTP instead of HTTPS	1
Hostname is an IP address	2
URL contains @ symbol	2
Contains sensitive keywords	1
More than four dot-separated hostname segments	1
Contains punycode label xn--	2
Hostname has three or more hyphens	1

3. How to Run the Program

1. Save the code as **phishing_url_detector.py**.
2. Open Command Prompt or Terminal.
3. Run the command: **python phishing_url_detector.py**.
4. Enter a URL to analyze it.
5. Type **quit** to exit.

4. Complete Python Program

```
from urllib.parse import urlparse
import ipaddress
import re


def analyze_url(url):
    """Check a URL for suspicious patterns using simple rules."""

    url = url.strip()

    if not url:
        return "INVALID URL", [], 0

    if not re.match(r"^[a-zA-Z][a-zA-Z0-9+.-]*://", url):
        url = "https://" + url

    try:
        parsed = urlparse(url)
        hostname = parsed.hostname

        if (
            parsed.scheme not in ("http", "https")
            or not hostname
            or any(char.isspace() for char in url)
```

```
        ):
            return "INVALID URL", [], 0

        score = 0
        indicators = []

        def flag(points, message):
            nonlocal score
            score += points
            indicators.append(message)

        if len(url) > 100:
            flag(1, "URL is unusually long")

        if parsed.scheme.lower() == "http":
            flag(1, "URL uses HTTP instead of HTTPS")

        try:
            ipaddress.ip_address(hostname)
            flag(2, "Hostname is an IP address")
        except ValueError:
            pass

        if "@" in parsed.netloc:
            flag(2, "URL contains an @ sign in its authority section")

        suspicious_words = (
            "login", "signin", "verify", "password",
            "account", "update", "confirm", "wallet"
        )

        found_words = [word for word in suspicious_words if word in url.lower()]

        if found_words:
            flag(1, "Sensitive-action keywords: " + ", ".join(found_words))

        if len(hostname.rstrip(".").split(".")) > 4:
            flag(1, "Hostname has many dot-separated segments")

        if any(label.startswith("xn--") for label in hostname.lower().split(".")):
            flag(2, "Hostname contains a punycode label")

        if hostname.count("-") >= 3:
            flag(1, "Hostname contains several hyphens")

        if score >= 3:
            result = "POTENTIALLY SUSPICIOUS"
        else:
            result = "NO OBVIOUS SUSPICIOUS INDICATORS"

        return result, indicators, score

    except (ValueError, UnicodeError):
        return "INVALID URL", [], 0


def main():
    print("=" * 48)
    print("       PHISHING URL DETECTION SYSTEM")
    print("=" * 48)
    print("Enter a URL to analyze.")
    print("Type 'quit' to exit.\n")

    while True:
        url = input("Enter URL: ").strip()

        if url.lower() in ("quit", "exit"):
            print("Exiting the detector. Goodbye!")
            break

        result, indicators, score = analyze_url(url)

        print("\n--- Analysis Report ---")
        print("Result:", result)
```

```
        print("Risk score:", score)

        if indicators:
            print("Indicators found:")
            for number, indicator in enumerate(indicators, start=1):
                print(f"  {number}. {indicator}")
        elif result == "INVALID URL":
            print("Please enter a valid HTTP or HTTPS URL.")
        else:
            print("No suspicious patterns matched the rules.")

        print("\nNote: This is a basic heuristic tool.")
        print("A suspicious result is not proof of phishing.")
        print("A low score does not prove a URL is safe.")
        print("-" * 48 + "\n")


if __name__ == "__main__":
    main()
```

5. Program Output

The following output shows three test cases: a suspicious phishing-like URL, a legitimate URL, and an IP-based URL.

```
========================================
       PHISHING URL DETECTION SYSTEM
========================================
Enter a URL to analyze.
Type 'quit' to exit.

Enter URL: http://secure-login@bank-verify.com

--- Analysis Report ---
Result: POTENTIALLY SUSPICIOUS
Risk score: 4

Indicators found:
  1. URL uses HTTP instead of HTTPS
  2. URL contains an @ sign in its authority section
  3. Sensitive-action keywords: login, verify

Note: This is a basic heuristic tool.
A suspicious result is not proof of phishing.
A low score does not prove a URL is safe.
----------------------------------------

Enter URL: https://www.google.com

--- Analysis Report ---
Result: NO OBVIOUS SUSPICIOUS INDICATORS
Risk score: 0

No suspicious patterns matched the rules.

Note: This is a basic heuristic tool.
A suspicious result is not proof of phishing.
A low score does not prove a URL is safe.
----------------------------------------

Enter URL: http://192.168.1.1/login

--- Analysis Report ---
Result: POTENTIALLY SUSPICIOUS
Risk score: 4

Indicators found:
  1. URL uses HTTP instead of HTTPS
  2. Hostname is an IP address
  3. Sensitive-action keywords: login

Note: This is a basic heuristic tool.
A suspicious result is not proof of phishing.
A low score does not prove a URL is safe.
----------------------------------------

Enter URL: quit
Exiting the detector. Goodbye!
```

6. Output Explanation

Test Case 1: http://secure-login@bank-verify.com was marked as potentially suspicious because it uses HTTP, contains an @ symbol, and includes the sensitive keywords login and verify.

Test Case 2: https://www.google.com was marked as having no obvious suspicious indicators because it uses HTTPS and has a normal domain structure.

Test Case 3: http://192.168.1.1/login was marked as potentially suspicious because it uses an IP address instead of a domain name, uses HTTP, and contains the keyword login.

7. Conclusion

This Python project successfully detects suspicious URLs using rule-based phishing indicators. It is a basic heuristic system, so a suspicious result is not final proof of phishing, and a low score does not guarantee that a URL is safe.